

A gamified introduction to Python Programming

Lecture 5

Looping with while statements

Matthieu DE MARI – Singapore University of Technology and Design



SINGAPORE UNIVERSITY OF
TECHNOLOGY AND DESIGN



Outline (Chapter 5)

- What is the **while** statement?
- What are **infinite loops**, why are they problematic and **how to stop them**?
- What is the **break** statement and how to use it in while statements?
- What are **good practices** when it comes to using while loops?
- (If time allows, let us discuss recursion!)

The **while** statement

The **while** statement is another type of conditional structure.

- **How it works:**

- If the Boolean condition specified for the **while** statement is **True**, then execute the block of code inside the **while** statement.
- If the Boolean condition is **False**, ignore the block of code in the **while** statement.



The **if** statement is the simplest conditional structure.

- **How it works:**

- If the Boolean condition specified for the **if** statement is **True**, then execute the block of code inside the **if** statement.
- If the Boolean condition is **False**, ignore the block of code in the **if** statement.
- Once we are done executing the code in **if** (or ignoring it), move on to the next (non-indented) line.

The **while** statement

The **while** statement is another type of conditional structure.

- **How it works:**

- If the Boolean condition specified for the **while** statement is **True**, then execute the block of code inside the **while** statement.
- If the Boolean condition is **False**, ignore the block of code in the **while** statement.
- Once we are done executing the code in **while**, move back to the while statement, and repeat until the condition is no longer True.



The **if** statement is the simplest conditional structure.

- **How it works:**

- If the Boolean condition specified for the **if** statement is **True**, then execute the block of code inside the **if** statement.
- If the Boolean condition is **False**, ignore the block of code in the **if** statement.
- Once we are done executing the code in **if** (or ignoring it), move on to the next (non-indented) line.

The **while** statement

The **while** statement is another type of conditional structure.

- **How it works:**

- If the Boolean condition specified for the **while** statement is **True**, then execute the block of code inside the **while** statement.
- If the Boolean condition is **False**, ignore the block of code in the **while** statement.
- Once we are done executing the code in **while**, move back to the while statement, and repeat until the condition is no longer True.

```
1  # Counting from 1 to 10
2  x = 0
3  print("Counting from 1 to 10...")
4  while(x<10):
5      x = x + 1
6      print(x)
7  print("Done!")
```

Counting from 1 to 10...

1

2

3

4

5

6

7

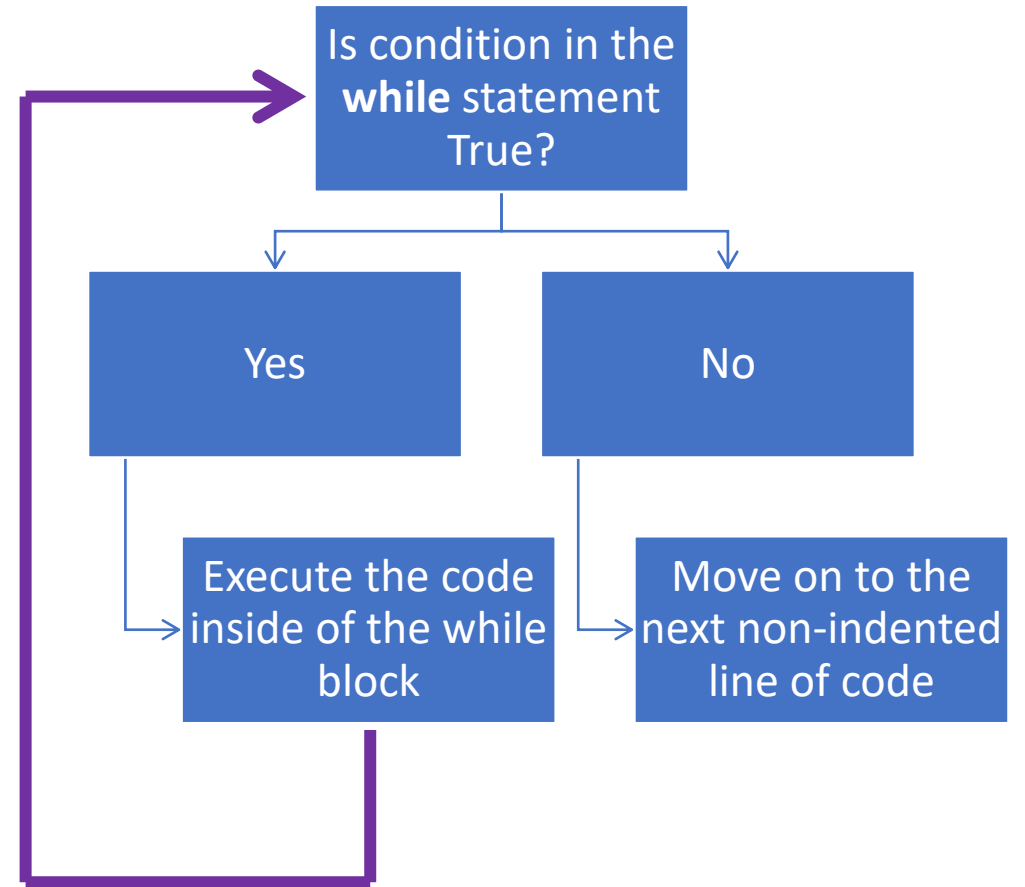
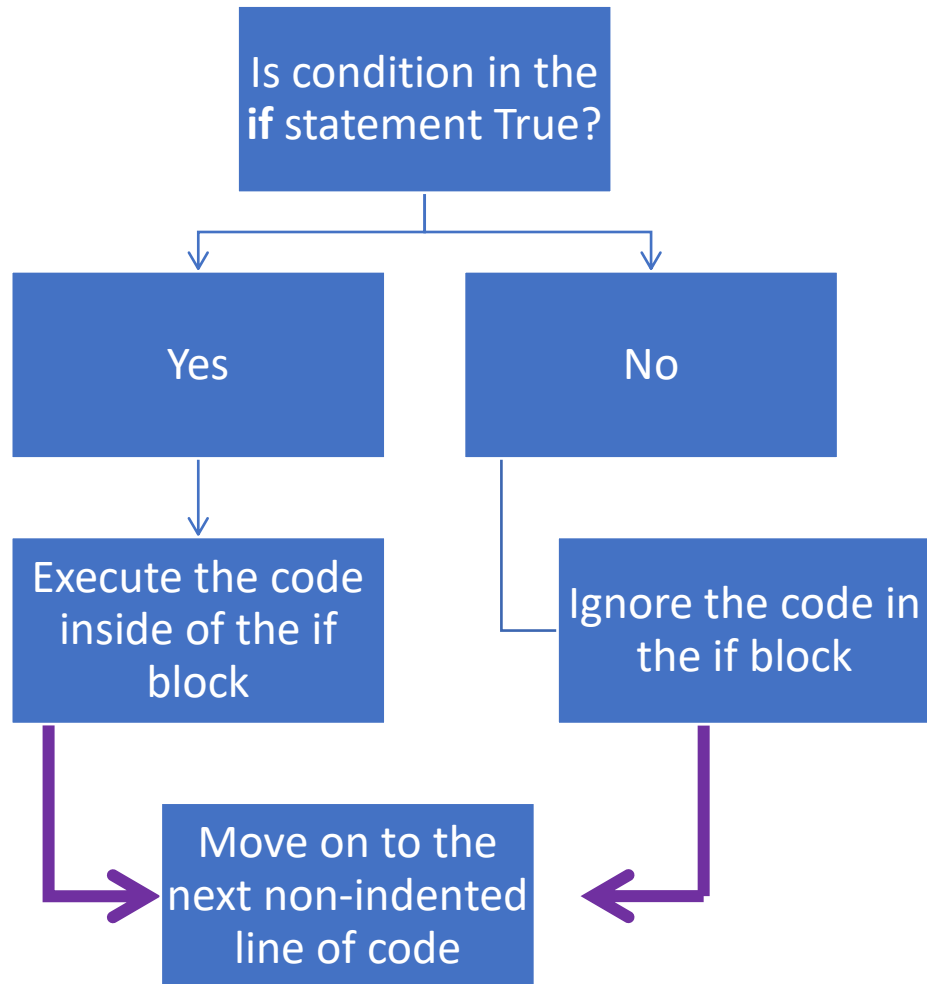
8

9

10

Done !

Architectures: **if** vs. **while**





Value of $x = 0$
Value of $x < 10 = \text{True}$
While loop **triggers**

- Execute $x = x + 1$
- Value of x is now 1
- Print value of x (1)

Value of $x = 1$
Value of $x < 10 = \text{True}$
While loop **triggers**

- Execute $x = x + 1$
- Value of x is now 2
- Print value of x (2)

Value of $x = 2$
Value of $x < 10 = \text{True}$
While loop **triggers**

- ...

**SOME MORE
ITERATIONS**

Value of $x = 10$
Value of $x < 10 = \text{False}$
While loop **stops**

- Do not execute code in while loop anymore
- Print "Done!"

```
1  # Counting from 1 to 10
2  x = 0
3  print("Counting from 1 to 10...")
4  while(x<10):
5      x = x + 1
6      print(x)
7  print("Done!")
```

Counting from 1 to 10...

1

2

3

4

5

6

7

8

9

10

Done!

Infinite loops

The **while** statement repeats a condition until it is no longer **True**.

This means that there should be a clear process that makes your condition no longer **True**, at some point.

```
1  # Counting from 1 to 10
2  x = 0
3  print("Counting from 1 to 10...")
4  while(x<10):
5      x = x + 1
6      print(x)
7  print("Done!")
```

Counting from 1 to 10...

1
2
3
4
5
6
7
8
9
10
Done !

Infinite loops

The **while** statement repeats a condition until it is no longer **True**.

This means that there should be a clear process that makes your condition no longer **True**, at some point.

Otherwise, big problem!

- The **while** block will keep on repeating indefinitely...
This is called an **infinite loop**.

```
In [4]: 1 # Counting from 1 to infinity
        2 x = 0
        3 while(x>=0):
        4     x = x + 1
        5     print(x)
        6 print("Done!")
```

```
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
```

Infinite loops and how to kill them

Infinite loops will keep on executing forever, unless

- ~~1. Your computer runs out of resources/memory and performs an emergency shutdown before exploding (**bad thing to do**),~~



You wrote an infinite while loop and kept it running for several hours. Your computer eventually ran out of memory and died.

We're just collecting some error info, and then we'll restart for you.

In the meantime, you should contemplate and reflect on your coding choices.

73% complete.



For more information about this issue and possible fixes, visit
<http://windows.com/stopcode>

If you call a support person, give them this info:
Stop code: CRITICAL_PROCESS_DIED

Infinite loops and how to kill them

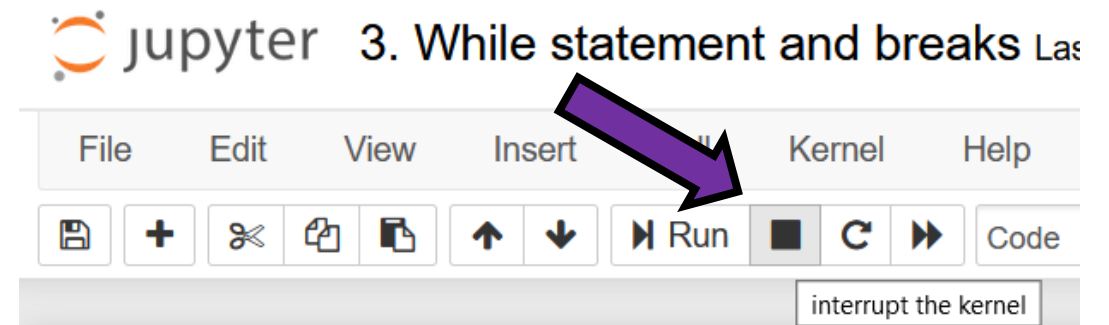
Infinite loops will keep on executing forever, unless

1. You decide to crash the program on purpose and kill the loop manually.

This is called a **keyboard interrupt**. It is done by pressing (repeatedly) **CTRL+C** (or **CMD+C** on mac), in console mode and most IDEs.

Or, by using the **stop button** on Jupyter.

```
Counting from 1 to infinity...
1
2
3
4
5
6
7
8
9
10
Traceback (most recent call last):
  File ".\infinite_loop.py", line 8, in <module>
    time.sleep(1)
KeyboardInterrupt
```



Infinite loops and how to kill them

Infinite loops will keep on executing forever, unless

1. You decide to crash the program on purpose and kill the loop manually.

This is called a **keyboard interrupt**. It is done by pressing (repeatedly) **CTRL+C** (or **CMD+C** on mac), in console mode and most IDEs.

Or, by using the **stop button** on Jupyter.

How it feels when you perform a keyword interrupt on your poor Python



Infinite loops and how to kill them

Infinite loops will keep on executing forever, unless

1. You decide to crash the program on purpose and kill the loop manually.

Infinite loops: the **break** statement

Infinite loops will keep on executing forever, unless

2. You use a **break** statement.

When encountered, the **break** statement will immediately end the current **while** loop.

The code then resumes its execution with the next line outside of the **while** block.

```
1 # Counting from 1 to 10, with a break
2 x = 0
3 while(True):
4     x = x + 1
5     print(x)
6     # If x has reached the value 10, break the while loop
7     if(x>=10):
8         break
9     # Careful!
10    print("This is DEAD CODE, because the break is reached before.")
11 print("Done!")
```

```
1
2
3
4
5
6
7
8
9
10
Done!
```

Standard **while** vs. infinite **while** + **break**

1. Standard **while** loop with condition in the while statement.

```
1 # Counting from 1 to 10
2 x = 0
3 print("Counting from 1 to 10...")
4 while(x<10):
5     x = x + 1
6     print(x)
7 print("Done!")
```

2. Infinite **while** loop with condition in an **if** statement, and **break** in the **if** block.

```
1 # Counting from 1 to 10, with a break
2 x = 0
3 while(True):
4     x = x + 1
5     print(x)
6     # If x has reached the value 10,
7     # break the while loop
8     if(x>=10):
9         break
10 print("Done!")
```

→ Both loops work and do the job, which one is better though?

Matt's Great Advice

Matt's Great Advice: Avoid the infinite loops, if possible (part 2).

Relying on an **infinite while** loop with a **break** is risky and should be avoided when possible.



Matt's Great Advice

Matt's Great Advice: Avoid the infinite loops, if possible (part 2).

Relying on an infinite **while** loop with a **break** is risky and should be avoided when possible.

It is often easily avoided, by using the Boolean expression of the **if** statement used for **break**, as the condition in the **while** statement.

```
1  # Counting from 1 to 10, with a break
2  x = 0
3  while(True):
4      x = x + 1
5      print(x)
6      # If x has reached the value 10,
7      # break the while loop
8      if(x>=10):
9          break
10 print("Done!")
```

```
1  # Counting from 1 to 10
2  x = 0
3  print("Counting from 1 to 10...")
4  while(x<10):
5      x = x + 1
6      print(x)
7  print("Done!")
```

Matt's Great Advice

Matt's Great Advice: Avoid the infinite loops, if possible (part 2).

Relying on an infinite **while** loop with a **break** is risky and should be avoided when possible.

It is often easily avoided, by using the Boolean expression of the **if** statement used for **break**, as the condition in the **while** statement.

Note: A few cases, however, require the use of a **break** statement.
For instance, **emergency shutdowns**.

```
1 while(True):  
2     print("All systems normal.")  
3     print("Running operations as expected.")  
4     if(overheating):  
5         print("Overheating detected.")  
6         print("Engaging emergency shutdown.")  
7         break
```

Some more advice before we practice

- **Understand the loop's purpose:** Before writing a loop, understand what you're trying to achieve. Is it iterating over a sequence until a condition is met, or repeating an action a certain number of times?
- **Write a simple pseudocode on paper:** Outline the logic of your loop in plain language or pseudocode. You NEED to clarify your thoughts before jumping into actual coding. Remember, this is step 1 for the life of a programmer! How are you supposed to translate into code a logic that you have not yet clarified begin with?
- **Set a clear stopping mechanism:** Make sure the loop has a well-defined condition that will eventually turn false, preventing infinite loops. Ask yourself, “Under what condition will this loop stop?”.

Some more advice before we practice

- **Start simple and refactor:** Do not try to figure the entire code at once and hope you will land the correct solution in one shot. It is often better to start simple, and then refactor/improve the loop, progressively adding more operations, in a controlled manner.
- **Review and simplify:** After getting your loop to work, revisit the code to see if it can be improved or be made more efficient. As you become more comfortable with loops, you will start noticing common loop patterns that can be reused in other problems.

Some more advice before we practice

- **Use and abuse the print!!!:**
Display the values of key variables at each iteration, to understand how your loop progresses and identify issues. You can always comment them/remove them later. Control your loops and do not trust your code!

→ Important Note: Call me for debugging + no prints = 2 SGD.

```
# This should run and produce the total sum  
# of all numbers between 1 and n  
def f(n):  
    total = 0  
    counter = 1  
    while counter < n:  
        total = total + counter  
        counter = counter + 1  
    return total  
  
# This should print 15 (= 1 + 2 + 3 + 4 + 5).  
print(f(5))
```

10



This does not produce the expected result, even though the code looked right?

Some more advice before we practice

- **Use and abuse the print!!!:**
Display the values of key variables at each iteration, to understand how your loop progresses and identify issues. You can always comment them/remove them later. Control your loops and do not trust your code!

→ Important Note: Call me for debugging + no prints = 2 SGD.

```
# This should run and produce the total sum  
# of all numbers between 1 and n  
def f(n):  
    total = 0  
    counter = 1  
    while counter < n:  
        print("-----")  
        total = total + counter  
        counter = counter + 1  
    return total  
  
# This should print 15 (= 1 + 2 + 3 + 4 + 5).  
print(f(5))
```

```
-----  
-----  
-----  
-----  
10
```

Adding a print of dashes at the beginning will show dashes at the beginning of each iteration. (Already shows that the loop did 4 iterations, instead of 5)

Some more advice before we practice

- **Use and abuse the print!!!:**
Display the values of key variables at each iteration, to understand how your loop progresses and identify issues. You can always comment them/remove them later. Control your loops and do not trust your code!

→ Important Note: Call me for debugging + no prints = 2 SGD.

```
# This should print 15 (= 1 + 2 + 3 + 4 + 5).  
print(f(5))
```

```
-----  
1 0  
2 1  
-----  
2 1  
3 3  
-----  
3 3  
4 6  
-----  
4 6  
5 10  
10
```

We can confirm that the counter variable went through the values 1, 2, 3, 4 at the beginning of the loops. The counter correctly increments by one at the end of the loop (Correct).

And that the total variable went through the values 0, 1, 3, 6, 10. This is also correct as it adds the value of counter at the beginning of the loop to the current value in the total variable (Correct).

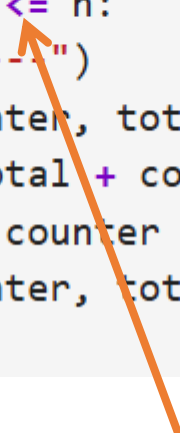
The issue is that we only perform 4 iterations and never add the counter value 5 to total.

Some more advice before we practice

- **Use and abuse the print!!!:**
Display the values of key variables at each iteration, to understand how your loop progresses and identify issues. You can always comment them/remove them later. Control your loops and do not trust your code!

→ Important Note: Call me for debugging + no prints = 2 SGD.

```
# This should run and produce the total sum  
# of all numbers between 1 and n  
def f(n):  
    total = 0  
    counter = 1  
    while counter <= n:  
        print("----")  
        print(counter, total)  
        total = total + counter  
        counter = counter + 1  
        print(counter, total)  
    return total
```



The fix is then to adjust the Boolean condition, replacing < with <=, to allow for one more iteration, the last one we were missing.

Some more advice before we practice

- **Use and abuse the print!!!:**
Display the values of key variables at each iteration, to understand how your loop progresses and identify issues. You can always comment them/remove them later. Control your loops and do not trust your code!

→ Important Note: Call me for debugging + no prints = 2 SGD.

```
# This should print 15 (= 1 + 2 + 3 + 4 + 5).  
print(f(5))
```

```
-----  
1 0  
2 1  
-----  
2 1  
3 3  
-----  
3 3  
4 6  
-----  
4 6  
5 10  
-----  
5 10  
6 15  
15
```

→ The prints will show that the adjustment worked! (Code is fixed!)

Matt's Great Advice

Matt's Great Advice: Avoid the infinite loops and dead code, by working on pen and paper first.

Infinite loops and **dead code**, unless created on purpose (!), usually follow from a **poor design** in your code.

Drawing a **structural diagram**, **before coding**, greatly helps figuring out the right structure for your code.

Use prints in your loops to see them in action and confirm their behavior!

Use pen and paper as well if it helps!



Matt's Great Advice

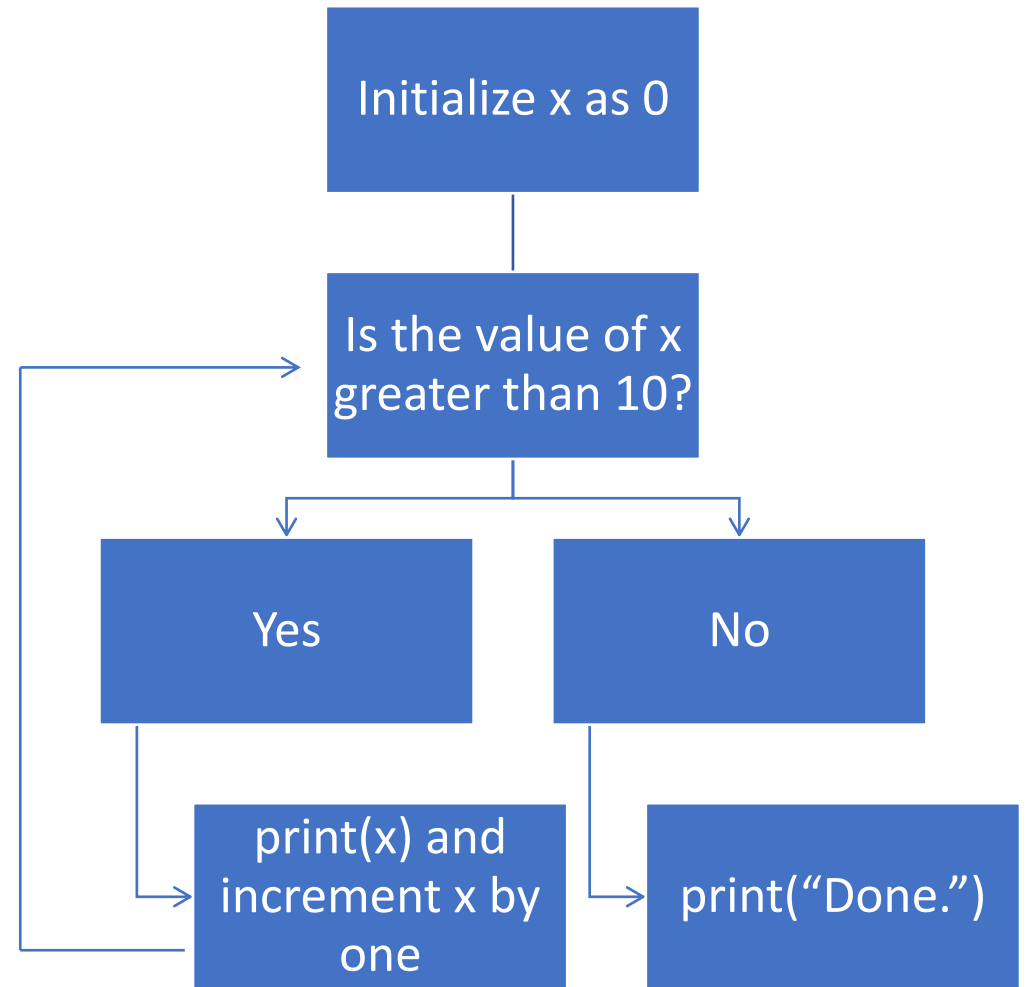
Matt's Great Advice: Avoid the infinite loops and dead code, by working on pen and paper first.

Infinite loops and **dead code**, unless created on purpose (!), usually follow from a **poor design** in your code.

Drawing a **structural diagram**, **before coding**, greatly helps figuring out the right structure for your code.

Use prints in your loops to see them in action and confirm their behavior!

Use pen and paper as well if it helps!



Example: diagram for our while loop, counting from 1 to 10.

Matt's Great Advice

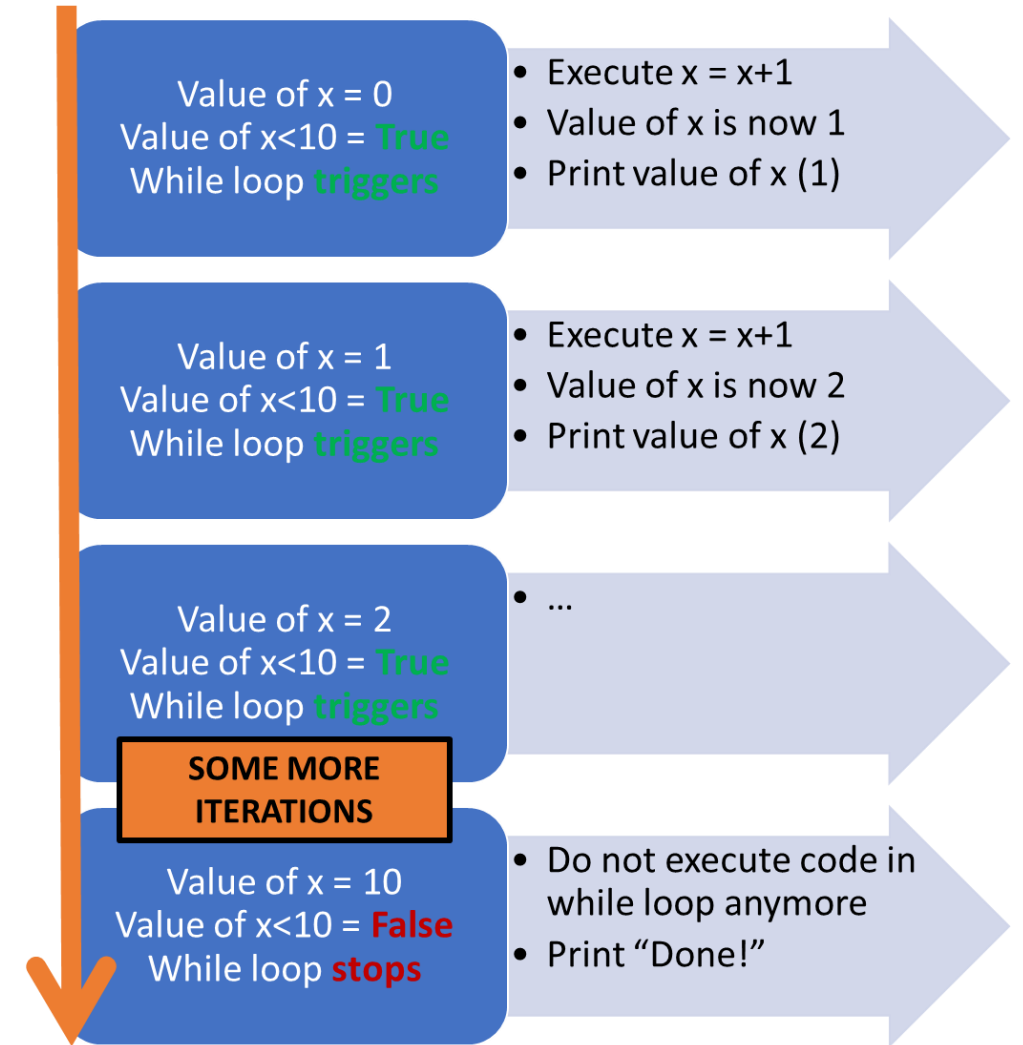
Matt's Great Advice: Avoid the infinite loops and dead code, by working on pen and paper first.

Infinite loops and **dead code**, unless created on purpose (!), usually follow from a **poor design** in your code.

Drawing a **structural diagram**, **before coding**, greatly helps figuring out the right structure for your code.

Use prints in your loops to see them in action and confirm their behavior!

Use pen and paper as well if it helps!



Example: diagram for our while loop, counting from 1 to 10.



Conclusion (Chapter 5)

- What is the **while** statement?
- What are **infinite loops**, why are they problematic and **how to stop them**?
- What is the **break** statement and how to use it in while statements?
- What are **good practices** when it comes to using while loops?
- (If time allows, let us discuss recursion!)

Quiz time!

Let's go!

slido

Please download and install the Slido app on all computers you use



What will be printed?

① Start presenting to display the poll results on this slide.

slido

Please download and install the Slido app on all computers you use



What will happen here?

① Start presenting to display the poll results on this slide.



Which statement should we use in the while loop, in order NOT to get stuck in a while loop?

① Start presenting to display the poll results on this slide.



Which of the following statements is NOT true about break statements in while loops?

① Start presenting to display the poll results on this slide.

Practice activities for **while/break**

Let us practice the **while/break** concepts a bit, with three activities.

- Activity 1: How much experience needed for level n?
- Activity 2: How many hits can you take?
- Activity 3: Guess the card game, v2.

(And as usual, some extra practice/extra challenges!)